Network Packet Analyzer – Project Documentation

By Rahul Kumar | Tech stack: Python, Scapy, Wireshark, SQLite

1. Overview

The Network Packet Analyzer is a Python-based tool that captures live network traffic, inspects each packet, and stores the important details in a database for later analysis. It works like a lightweight version of Wireshark: it listens on a network interface, breaks every packet into its layers, and records who is talking to whom, over which port, and using which protocol.

The tool also produces a traffic summary (for example, how many packets were TCP, UDP, ICMP or DNS), which helps with basic network monitoring and troubleshooting.

2. Objectives

- Capture live network packets using Scapy.
- Identify the protocol of each packet: TCP, UDP, ICMP and DNS.
- Extract source IP, destination IP, source port, destination port and protocol.
- Log every packet in SQLite (and optionally export to CSV) for later analysis.
- Generate a summary of protocol distribution and packet statistics.
- Validate the results by comparing them with Wireshark.

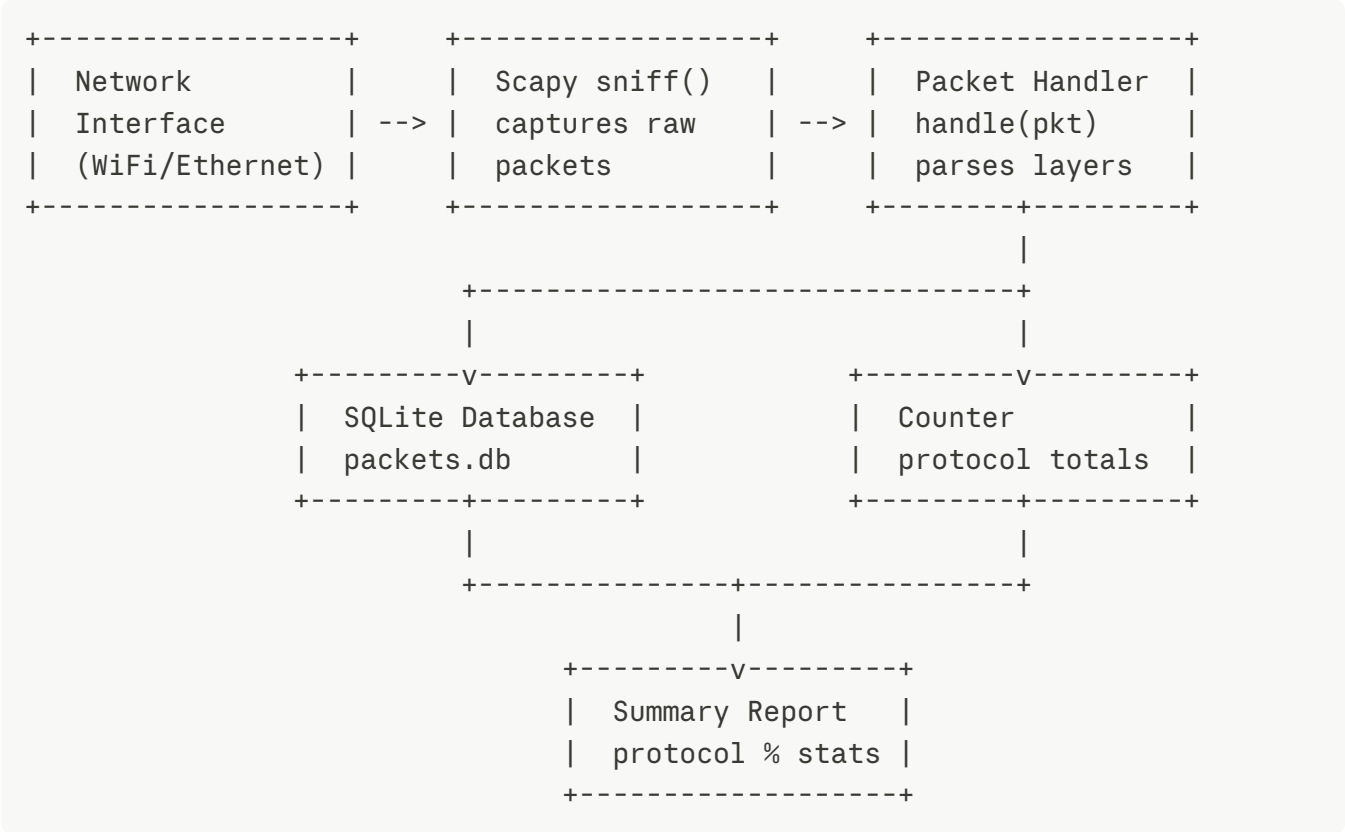
3. Technology Stack

Component	Purpose
Python 3	Main programming language
Scapy	Packet capture, parsing and layer access
SQLite	Local database for packet logs
CSV	Optional export format for spreadsheets

Component	Purpose
Wireshark	Used to cross-check and verify captured traffic
Npcap (Windows) / libpcap (Linux, Mac)	Low-level capture driver used by Scapy

4. System Architecture

The tool follows a simple pipeline: capture, parse, store, summarize.



Components

1. **Capture layer:** Scapy's `sniff()` listens on the network interface and passes every packet to a callback function.
2. **Parsing layer:** The `handle()` function checks which layers the packet contains (IP, TCP, UDP, ICMP, DNS) and extracts the fields.
3. **Storage layer:** Each parsed packet is inserted into an SQLite table.
4. **Analysis layer:** A Counter keeps protocol totals, and a summary is printed when capture ends.

5. Project Structure

```
network-packet-analyzer/
├── analyzer.py          # main program
├── requirements.txt     # dependencies (scapy)
├── README.md            # project description
├── .gitignore           # ignores *.db, *.csv, __pycache__/
└── screenshots/
    └── output.png       # terminal output screenshot
```

6. Installation and Setup

1. Install Python 3.8 or newer.
2. Install the dependency:

```
pip install scapy
```

3. On Windows, install Npcap from npcap.com (installing Wireshark also installs it). On Linux and Mac no extra driver is needed.
4. Run the script with administrator rights, because packet capture needs raw socket access:

```
sudo python3 analyzer.py
```

While it runs, open a few websites or run `ping google.com` so there is traffic to capture.

7. Source Code

```
from scapy.all import sniff, IP, TCP, UDP, ICMP, DNS, DNSQR
from collections import Counter
import sqlite3
from datetime import datetime

conn = sqlite3.connect("packets.db")
conn.execute("""CREATE TABLE IF NOT EXISTS packets(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    time TEXT, src TEXT, dst TEXT, proto TEXT,
    sport INTEGER, dport INTEGER, info TEXT)""")
```

```
counts = Counter()

def handle(pkt):
    if IP not in pkt:
        return
    src, dst = pkt[IP].src, pkt[IP].dst
    sport = dport = None
    info = ""

    if TCP in pkt:
        proto, sport, dport = "TCP", pkt[TCP].sport, pkt[TCP].dport
    elif UDP in pkt:
        proto, sport, dport = "UDP", pkt[UDP].sport, pkt[UDP].dport
    elif ICMP in pkt:
        proto = "ICMP"
    else:
        proto = "OTHER"

    if pkt.haslayer(DNS) and pkt.haslayer(DNSQR):
        proto = "DNS"
        info = pkt[DNSQR].qname.decode(errors="ignore")

    counts[proto] += 1
    conn.execute("INSERT INTO packets(time,src,dst,proto,sport,dport,info)
VALUES(?,?,?,?,?,?,?)",
                 (datetime.now().isoformat(), src, dst, proto, sport, dport,
info))
    conn.commit()
    print(f"{proto:5} {src}:{sport} -> {dst}:{dport} {info}")

sniff(prn=handle, count=100, store=False)

print("\n--- Summary ---")
total = sum(counts.values())
for p, c in counts.most_common():
    print(f"{p}: {c} packets ({c/total*100:.1f}%)")
```

8. Code Walkthrough

Imports. `sniff` captures packets. `IP`, `TCP`, `UDP`, `ICMP`, `DNS` and `DNSQR` are Scapy layer classes used to inspect packets. `Counter` counts protocols, and `sqlite3` stores the logs.

Database setup. The script opens (or creates) `packets.db` and creates a `packets` table if it does not already exist.

The `handle(pkt)` function. It runs once for every captured packet:

1. It skips anything without an IP layer.
2. It reads the source and destination IP addresses.
3. It checks whether the packet is TCP, UDP or ICMP and reads the port numbers where they exist.
4. If the packet carries a DNS query, it labels the protocol as DNS and saves the domain name that was looked up.
5. It updates the protocol counter, inserts a row into SQLite, and prints a one-line description.

The `sniff()` call. `prn=handle` tells Scapy to call `handle` for each packet. `count=100` stops after 100 packets. `store=False` avoids keeping packets in memory, so long captures do not use up RAM.

The summary. After capture ends, the script prints each protocol, its packet count and its percentage of total traffic.

9. Database Schema

Table: `packets`

Column	Type	Description
id	INTEGER (primary key)	Auto-increment row ID
time	TEXT	Timestamp when the packet was captured
src	TEXT	Source IP address
dst	TEXT	Destination IP address
proto	TEXT	TCP, UDP, ICMP, DNS or OTHER
sport	INTEGER	Source port (empty for ICMP)
dport	INTEGER	Destination port (empty for ICMP)
info	TEXT	Extra information, such as the DNS domain queried

Useful queries for analysis:

```
-- Packets per protocol
SELECT proto, COUNT(*) FROM packets GROUP BY proto;

-- Top 5 destination IPs
SELECT dst, COUNT(*) AS n FROM packets GROUP BY dst ORDER BY n DESC LIMIT 5;

-- All DNS lookups
SELECT time, info FROM packets WHERE proto = 'DNS';
```

10. Sample Output

The block below is a sample of what the terminal output looks like. Your exact lines will differ because they depend on your own network traffic. Replace this with output from your own run.

```
DNS   192.168.1.5:54213 -> 8.8.8.8:53 www.google.com.
UDP   192.168.1.5:54213 -> 8.8.8.8:53
TCP   192.168.1.5:51822 -> 142.250.183.4:443
TCP   142.250.183.4:443 -> 192.168.1.5:51822
ICMP  192.168.1.5:None -> 142.250.183.4:None
ICMP  142.250.183.4:None -> 192.168.1.5:None

--- Summary ---
TCP: 68 packets (68.0%)
UDP: 14 packets (14.0%)
DNS: 10 packets (10.0%)
ICMP: 8 packets (8.0%)
```

11. Screenshots

Add your own screenshots here before sharing the document. Take them from your real run so they match your code.

- **Screenshot 1: Live capture.** Terminal showing packet lines while the script runs.
- **Screenshot 2: Summary.** Terminal showing the protocol distribution at the end.
- **Screenshot 3: Database.** The `packets` table opened in DB Browser for SQLite.

- **Screenshot 4: Wireshark comparison.** Wireshark capture of the same traffic, to show your results match.

12. Protocol Reference

Protocol	What it does	Common ports
TCP	Reliable, connection-based delivery with a 3-way handshake (SYN, SYN-ACK, ACK)	80 (HTTP), 443 (HTTPS), 22 (SSH)
UDP	Fast, connectionless delivery with no delivery guarantee	53 (DNS), 123 (NTP)
ICMP	Error and diagnostic messages, used by ping	none (no ports)
DNS	Converts domain names into IP addresses	53

13. Use Cases

- Learning how network protocols behave in real traffic.
- Basic network monitoring: seeing which protocols dominate the traffic.
- Troubleshooting: checking whether DNS queries are leaving the machine or whether pings get replies.
- Spotting unusual activity, such as one IP address sending a very large number of packets.

14. Limitations

- It only analyses IP traffic, so non-IP frames such as ARP are skipped.
- It does not decrypt HTTPS, so it sees addresses and ports but not page content.
- It does not detect threats by itself; it only records and summarizes.
- Writing to the database on every packet can slow the tool down on very busy networks.
- Only IPv4 is handled in this version.

15. Future Improvements

- Add IPv6 and ARP support.
- Export logs to CSV from the command line.
- Add filters, for example capture only port 443 or only one IP.
- Show a live chart of protocol distribution.
- Add alerts for suspicious behaviour, such as a possible port scan.
- Batch database writes for better performance.

16. Interview Questions and Answers

What does this project do? It captures live network packets with Scapy, extracts IPs, ports and protocols, stores them in SQLite, and prints a protocol summary.

Why Scapy instead of Wireshark? Wireshark is a ready-made tool. Scapy lets me write code to capture and process packets, so I can automate analysis and store results myself.

What is the difference between TCP and UDP? TCP is connection-based and reliable, with handshakes and retransmission. UDP is connectionless and faster but does not guarantee delivery.

What is the TCP 3-way handshake? The client sends SYN, the server replies SYN-ACK, and the client sends ACK. After that the connection is established.

How does DNS work? The client asks a DNS server for the IP address of a domain, usually over UDP port 53, and the server replies with the address.

Why does the script need admin rights? Capturing packets needs raw access to the network interface, which the operating system restricts to privileged users.

How did you verify your results? I compared the packets and protocol counts with a Wireshark capture of the same traffic.

17. Conclusion

This project shows how network traffic is built from layers and how a small Python program can capture, classify and store it. It builds practical knowledge of TCP/IP, DNS, packet parsing and basic network monitoring, and it is a base that can be extended with filtering, alerts and visual dashboards.